ZÁRÓDOLGOZAT

Miskolc

2026

ADATKEZELŐ WEBALKALMAZÁS

Készítette: Jobbágy Fanni és Jakubinyi Máté

Miskolc

2026

TARTALOMJEGYZÉK (vizsgabarát)

1. Bevezetés
2. A projekt célja és követelményrendszere
3. Alkalmazott fejlesztési módszertan
4. Felhasznált technológiák részletes bemutatása
5. A rendszer architektúrája
6. Backend megvalósítás (Laravel)
7. Frontend megvalósítás
8. Adatbázis-tervezés és megvalósítás
9. Funkcionális leírás (use case-ek)
10. Felhasználói dokumentáció
11. Hibakezelés és validáció
12. Tesztelési dokumentáció
13. Verziókezelés és munkafolyamat
14. Biztonsági megoldások
15. Telepítési útmutató
16. Továbbfejlesztési lehetőségek
17. Összegzés
18. Forrásjegyzék

1. BEVEZETÉS

A szoftverfejlesztés napjaink egyik legdinamikusabban fejlődő területe, amely szinte minden iparágban kulcsszerepet tölt be. A digitalizáció előretörésével egyre nagyobb igény mutatkozik olyan informatikai rendszerek iránt, amelyek képesek hatékonyan kezelni adatokat, automatizálni folyamatokat, valamint támogatni a felhasználók mindennapi munkáját.

Jelen dokumentáció az **Oktáv Szoftverfejlesztő – tesztelő képzés vizsgaremekének** részeként készült. A projekt célja egy **webalapú alkalmazás** elkészítése, amely a modern webfejlesztési elveknek megfelelően, MVC architektúrára épülve valósít meg adatkezelési funkciókat.

A fejlesztés során kiemelt figyelmet kaptak:

- az átlátható kódstruktúra
- a karbantarthatóság
- az adatbiztonság
- a felhasználóbarát megjelenés
- valamint a tesztelhetőség

Az alkalmazás Laravel keretrendszer segítségével készült, amely napjaink egyik legelterjedtebb PHP alapú webes frameworkje.

2. A PROJEKT CÉLJA ÉS KÖVETELMÉNYRENDSZERE

2.1 A projekt célja

A projekt elsődleges célja egy olyan webalkalmazás megvalósítása volt, amely:

- CRUD (Create, Read, Update, Delete) műveleteket valósít meg
- adatbázissal kommunikál
- strukturált MVC elrendezést használ
- vizsgafeladatként teljes mértékben megfelel a képzés követelményeinek

A rendszer lehetőséget biztosít adatok rögzítésére, módosítására, törlésére és listázására webes felületen keresztül.

2.2 Funkcionális követelmények

- Adatok kezelése adatbázisban
- Webes felhasználói felület
- Hibakezelés és validáció
- Strukturált routing rendszer

2.3 Nem funkcionális követelmények

- Áttekinthető kód
- Karbantarthatóság
- Reszponzív megjelenés
- Biztonságos adatkezelés

3. ALKALMAZOTT FEJLESZTÉSI MÓDSZERTAN

A fejlesztés során iteratív fejlesztési megközelítés került alkalmazásra. Ez lehetővé tette, hogy az egyes funkciók külön egységként kerüljenek megvalósításra, majd folyamatosan továbbfejlesztésre és tesztelésre.

A fejlesztési lépések:

1. **Követelmények elemzése**
2. **Adatbázis tervezés**
3. **Backend megvalósítás**
4. **Frontend felület kialakítása**
5. **Tesztelés**
6. **Dokumentáció elkészítése**

4. FELHASZNÁLT TECHNOLÓGIÁK részletes bemutatása

4.1 PHP

A PHP egy szerveroldali programozási nyelv, amelyet széles körben használnak webalkalmazások készítésére. A Laravel keretrendszer is PHP-ra épül, így a backend teljes mértékben PHP alapú.

4.2 Laravel keretrendszer

Részletesen:

- MVC architektúra
- Routing
- Controller logika
- Eloquent ORM
- Migrációk

4.3 MySQL adatbázis

- relációs modell
- táblakapcsolatok
- elsődleges idegen kulcsok
- adatkonzisztencia

4.4 Blade Template Engine

- layoutok
- komponensek
- dinamikus adatmegjelenítés

5. FELHASZNÁLT TECHNOLÓGIÁK – RÉSZLETES ISMERTETÉS

5.1 PHP – Szerveroldali programozási nyelv

A PHP (Hypertext Preprocessor) egy széles körben elterjedt, nyílt forráskódú szerveroldali programozási nyelv, amelyet kifejezetten webes alkalmazások fejlesztésére terveztek. A PHP kód a szerveren fut le, és a futás eredményeként generált HTML kerül visszaküldésre a kliens számára.

A projekt backend része teljes egészében PHP nyelven készült. A PHP előnyei közé tartozik:

- egyszerű szintaxis
- széles körű dokumentáció
- nagy közösségi támogatás
- könnyű integráció adatbázisokkal

A nyelv különösen alkalmas kisebb és közepes méretű adatkezelő rendszerekhez, mint amilyen jelen vizsgaprojekt is.

5.2 Laravel keretrendszer

A Laravel egy modern PHP keretrendszer, amely az MVC (Model–View–Controller) tervezési mintát követi. A keretrendszer célja, hogy leegyszerűsítse a webalkalmazások fejlesztését, miközben strukturált és karbantartható kódbázist biztosít.

Az MVC szerepe a projektben

- **Model**: az adatbázissal való kommunikációért felel
- **View**: a felhasználói felület megjelenítését végzi
- **Controller**: az üzleti logikát irányítja

Ez az elkülönítés különösen fontos a tesztelhetőség és az átláthatóság szempontjából, ami az Oktáv képzésen elvárt szakmai szinthez illeszkedik.

Laravel főbb komponensei:

- Routing rendszer
- Controller-ek
- Eloquent ORM

- Migrációk
- Beépített validáció
- CSRF védelem

5.3 MySQL – Adatbázis-kezelő rendszer

A projekt adatai MySQL relációs adatbázisban kerülnek tárolásra. A MySQL megbízható, gyors és jól skálázható megoldást kínál webalkalmazások számára.

Az adatbázis használatának előnyei:

- relációs szerkezet
- kulcs-kapcsolatok kezelése
- adatkonzisztencia biztosítása
- SQL alapú lekérdezések

A Laravel Eloquent ORM segítségével az adatbázis-műveletek objektumorientált módon valósultak meg.

5.4 Blade Template Engine

A Blade a Laravel saját sablonmotorja, amely lehetővé teszi a dinamikus adatok egyszerű megjelenítését HTML oldalakon.

Főbb jellemzői:

- layout öröklődés
- vezérlési szerkezetek (@if, @foreach)
- kódismétlés csökkentése
- tiszta HTML kimenet

A Blade használatával a frontend kód jól elkülönül a backend logikától.

5.5 HTML, CSS és Bootstrap

A felhasználói felület megjelenítéséhez HTML és CSS került felhasználásra.
A reszponzív dizájn kialakítását a Bootstrap keretrendszer segítette.

A Bootstrap előnyei:

- mobilbarát kialakítás
- előre definiált komponensek
- egységes megjelenés
- gyors fejlesztés

6. BACKEND MEGVALÓSÍTÁS (LARAVEL)

A backend megvalósítása során a Laravel keretrendszer alapértelmezett struktúrája került alkalmazásra. A rendszer logikai működéséért a Controller réteg felel.

6.1 Routing

A routing rendszer határozza meg, hogy egy adott HTTP kérés mely kontrollerhez kerüljön. A projektben a Laravel routes/web.php fájl tartalmazza az útvonalakat.

6.2 Controller-ek szerepe

A controller-ek:

- fogadják a felhasználói kéréseket
- feldolgozzák az adatokat
- meghívják a megfelelő modelleket
- visszaküldik a nézeteket

6.3 Eloquent ORM

Az Eloquent ORM lehetővé teszi az adatbázis-műveletek objektumorientált megvalósítását.
Ez nagymértékben csökkenti a hibalehetőségeket és javítja az olvashatóságot.

7. Frontend megvalósítás

7.1 A frontend szerepe az alkalmazásban

A frontend az alkalmazás azon része, amellyel a felhasználó közvetlenül kapcsolatba lép. Feladata, hogy az adatkezelési funkciókat átlátható, könnyen használható és esztétikus formában jelenítse meg.
A frontend megfelelő kialakítása nagymértékben hozzájárul a felhasználói élményhez és a rendszer hatékony használatához.

Jelen projekt esetében kiemelt szempont volt:

- az egyszerű, letisztult megjelenés
- az egyértelmű navigáció
- az adatbeviteli hibák minimalizálása
- a reszponzív megjelenés biztosítása

A frontend kialakítása során a cél nem egy bonyolult grafikai megoldás volt, hanem egy **vizsgára optimalizált, funkcionális felület** kialakítása.

7.2 Alkalmazott frontend technológiák

A frontend fejlesztéséhez az alábbi technológiák kerültek felhasználásra:

- **HTML (HyperText Markup Language)** – a weboldalak szerkezetének kialakítására
- **CSS (Cascading Style Sheets)** – az oldal megjelenésének formázására
- **Bootstrap** – reszponzív dizájn és komponensek biztosítására
- **Blade template engine** – dinamikus tartalom megjelenítésére Laravel környezetben

Ezek a technológiák együttesen biztosítják, hogy a felület modern, jól strukturált és könnyen karbantartható legyen.

7.3 Blade sablonrendszer használata

A Laravel Blade sablonrendszere lehetővé teszi a frontend és a backend logika hatékony szétválasztását.
A Blade használatával elkerülhető a kódismétlés, valamint áttekinthetőbbé válik a weboldalak felépítése.

A Blade alkalmazásának előnyei:

- közös layout fájl használata
- dinamikus adatmegjelenítés
- ciklusok és feltételek egyszerű kezelése
- karbantartható sablonszerkezet

Az alkalmazás minden oldala egy közös alapelrendezést (layoutot) használ, amely tartalmazza:

- a fejlécet
- a navigációs menüt
- a tartalmi részt
- az esetleges láblécet

Ez biztosítja az egységes megjelenést az alkalmazás teljes felületén.

7.4 Felhasználói oldalak felépítése

A frontend több különböző funkcionális oldalt tartalmaz, amelyek mindegyike az adatkezelési folyamat egy-egy lépését szolgálja.

Főbb oldaltípusok:

- főoldal
- adatlista oldal
- új adat felvitele
- adat módosítása
- visszajelző üzenetek megjelenítése

Az oldalak logikusan felépített menürendszeren keresztül érhetők el, így a felhasználó gyorsan megtalálja a kívánt funkciót.

7.5 Űrlapok és adatbevitel

Az adatbevitel webes űrlapokon keresztül történik.
Az űrlapok célja az, hogy a felhasználó strukturált módon tudja megadni az adatokat, miközben a rendszer segíti a helyes kitöltést.

Az űrlapok jellemzői:

- kötelező mezők megjelölése
- jól elkülönített mezők
- egyértelmű feliratok
- visszajelzés hibás adatbevitel esetén

Az adatok elküldése után a rendszer minden esetben visszajelzést ad a művelet eredményéről, amely hozzájárul a felhasználó tájékoztatásához.

7.6 Validáció megjelenítése a frontend oldalon

A frontend fontos feladata a hibás bevitel kezelése és annak megjelenítése a felhasználó számára.
A validációs hibák vizuálisan jól elkülönülnek, így azonnal felismerhetőek.

A frontend oldalon:

- hibaüzenetek jelennek meg hibás adat esetén
- sikeres művelet esetén megerősítő üzenet látható
- a felhasználó nem veszíti el az addig bevitt adatokat

Ez a megoldás csökkenti a felhasználói hibák számát és növeli az alkalmazás használhatóságát.

7.7 Reszponzív megjelenés

A frontend kialakítása során fontos szempont volt, hogy az alkalmazás különböző képernyőméreteken is megfelelően jelenjen meg.
A Bootstrap keretrendszer segítségével a weboldalak automatikusan alkalmazkodnak az eszköz méretéhez.

Ez lehetővé teszi, hogy az alkalmazás:

- asztali számítógépen
- laptopon
- tableten
- mobiltelefonon

is használható legyen, anélkül hogy a funkcionalitás sérülne.

7.8 Felhasználói élmény (UX) szempontok

A frontend tervezése során a felhasználói élmény kiemelt szerepet kapott.
A cél az volt, hogy az alkalmazás használata intuitív legyen, és ne igényeljen hosszabb betanulást.

Ennek érdekében:

- az oldalak logikus sorrendben követik egymást
- a gombok jól láthatók
- a visszajelzések egyértelműek
- a felület nem tartalmaz felesleges elemeket

Ez a megközelítés különösen fontos olyan rendszereknél, ahol a felhasználók eltérő technikai háttérrel rendelkeznek.

7.9 A frontend szerepe a vizsgaprojektben

A frontend megvalósítása jól szemlélteti a hallgató képességeit a webes felületek kialakítása terén.
A projekt frontend része megfelel a szoftverfejlesztő–tesztelő képzés elvárásainak, mivel:

- átlátható
- funkcionális
- tesztelhető
- karbantartható

A frontend sikeresen támogatja a backend működését, és teljes mértékben ellátja az alkalmazással szemben támasztott elvárásokat.

9. ADATBÁZIS TERVEZÉS

Az adatbázis tervezése során fontos szempont volt az egyszerűség és az átláthatóság. A táblák migrációk segítségével jöttek létre, amely biztosítja:

- az adatbázis verziózhatóságát
- az újratelepíthetőséget
- a hibamentes struktúrát

9.1 Táblák és kapcsolatok

- egyedek egyértelmű azonosítása
- elsődleges kulcsok használata
- relációk kialakítása

8. Adatbázis-tervezés és megvalósítás

8.1 Az adatbázis szerepe az alkalmazásban

Az adatbázis az alkalmazás egyik legfontosabb része, mivel biztosítja az adatok tartós tárolását és visszakereshetőségét.
A webalkalmazás működésének alapját az adatbázisban tárolt adatok képezik, amelyekhez a felhasználó a frontend felületen keresztül fér hozzá.

Jelen projekt esetében az adatbázis feladata:

- a rögzített adatok tárolása
- az adatok módosításának és törlésének kezelése
- az adatok strukturált és biztonságos kezelése
- a backend és frontend közötti adatkapcsolat biztosítása

Az adatbázis tervezése során kiemelt figyelmet kapott az egyszerűség, az átláthatóság és a bővíthetőség.

8.2 Alkalmazott adatbázis-kezelő rendszer

Az alkalmazás **MySQL** relációs adatbázis-kezelő rendszert használ.
A MySQL széles körben elterjedt, megbízható és jól integrálható Laravel környezetben, ezért ideális választás volt a vizsgaprojekt számára.

A relációs adatbázis előnyei:

- táblák közötti kapcsolatok kezelése
- adatkonzisztencia biztosítása
- elsődleges és idegen kulcsok használata
- strukturált adatmodell

A Laravel az adatbázissal az **Eloquent ORM** segítségével kommunikál, amely leegyszerűsíti az adatkezelési műveleteket.

8.3 Adatbázis-tervezési szempontok

Az adatbázis tervezése során az alábbi alapelvek kerültek figyelembevételre:

- **Normalizált szerkezet**
  Az adatok ismétlődésének minimalizálása érdekében a táblák normalizált formában kerültek kialakításra.
- **Egyértelmű azonosítás**
  Minden rekord egyedi azonosítóval (elsődleges kulccsal) rendelkezik.
- **Logikus adattípusok**
  Az egyes mezők adattípusai az adatok jellegéhez igazodnak.
- **Bővíthetőség**
  Az adatbázis szerkezete lehetőséget biztosít további táblák és kapcsolatok hozzáadására.

Ezek a szempontok biztosítják a rendszer hosszú távú fenntarthatóságát.

8.4 Táblák és mezők felépítése

Az adatbázis több táblából áll, amelyek mindegyike egy adott entitást reprezentál.
A táblák mezői a tárolt adatok logikai csoportosítását követik.

Egy tipikus tábla tartalmaz:

- egyedi azonosítót
- leíró adatmezőket
- időbélyegeket (létrehozás, módosítás ideje)

Az időbélyegek segítségével nyomon követhető az adatok változása, amely a tesztelés és hibakeresés során is hasznos.

8.5 Kapcsolatok az egyes táblák között

A rendszerben lévő táblák logikai kapcsolatban állnak egymással.
A kapcsolatok kialakítása idegen kulcsok segítségével történt, amelyek biztosítják az adatok közötti összefüggések érvényességét.

A táblák közötti kapcsolatok előnyei:

- adatkonzisztencia
- redundancia csökkentése
- strukturált adatkezelés

A Laravel Eloquent ORM automatikusan kezeli ezeket a kapcsolatokat, így a backend oldalon az adatlekérdezések egyszerűbbé válnak.

8.6 Migrációk használata

Az adatbázis létrehozása és módosítása **Laravel migrációk** segítségével történt.
A migrációk lehetővé teszik az adatbázis szerkezetének verziókezelését.

A migrációk előnyei:

- az adatbázis újratelepíthető
- az adatbázis szerkezete dokumentálva van
- verziókövetés valósítható meg
- csapatmunkát is támogat

Ez a megoldás megfelel a modern fejlesztési gyakorlatoknak, és kifejezetten előnyös vizsgaprojekt esetén is.

8.7 Adatkezelés Eloquent ORM használatával

Az adatbázis-műveletek a Laravel **Eloquent ORM** rétegén keresztül valósulnak meg.
Az ORM lehetővé teszi, hogy az adatbázis-rekordokat objektumként kezeljük, ezáltal javítva a kód olvashatóságát.

Az Eloquent használatának előnyei:

- kevesebb SQL kód
- objektumorientált megközelítés
- egyszerű CRUD műveletek
- kapcsolatok kezelése programozói szinten

Ez a megoldás jól illeszkedik az MVC architektúrába.

8.8 Adatbázis-biztonság

Az adatbázis biztonsága kiemelt szerepet kapott a fejlesztés során.
A Laravel beépített megoldásai hozzájárulnak az adatok védelméhez.

Biztonsági megoldások:

- validáció a mentés előtt
- SQL injection elleni védelem
- ellenőrzött adatbevitel
- strukturált adatkezelés

Ezek a megoldások csökkentik a biztonsági kockázatokat és növelik a rendszer megbízhatóságát.

8.9 Az adatbázis szerepe a tesztelés során

A tesztelés során az adatbázis működése külön ellenőrzésre került.
Vizsgálva lett, hogy:

- az adatok helyesen kerülnek-e mentésre
- módosítás során az adatok megfelelően frissülnek-e
- törlés esetén az adatbázis konzisztens marad-e

A teszteredmények igazolták, hogy az adatbázis-kezelés stabil és megbízható módon működik.

8.10 Összegzés

Az adatbázis-tervezés és megvalósítás során egy jól strukturált, bővíthető és biztonságos adatkezelő megoldás készült.
Az alkalmazott technológiák és tervezési elvek megfelelnek a szoftverfejlesztő–tesztelő képzés szakmai követelményeinek.

Az elkészült adatbázis stabil alapot biztosít az alkalmazás működéséhez és további fejlesztésekhez is.

9. ADATBÁZIS TERVEZÉS

Az adatbázis tervezése során fontos szempont volt az egyszerűség és az átláthatóság. A táblák migrációk segítségével jöttek létre, amely biztosítja:

- az adatbázis verziózhatóságát
- az újratelepíthetőséget
- a hibamentes struktúrát

9.1 Táblák és kapcsolatok

- egyedek egyértelmű azonosítása
- elsődleges kulcsok használata
- relációk kialakítása

10. Felhasználói dokumentáció

10.1 A felhasználói dokumentáció célja

A felhasználói dokumentáció célja, hogy részletes és közérthető útmutatást nyújtson az alkalmazás használatához.
Ez a fejezet nem fejlesztőknek, hanem **végfelhasználóknak** készült, ezért a szakmai megfogalmazás helyett az egyszerű, érthető leírás került előtérbe.
A dokumentáció segítségével a felhasználó:

- megismeri az alkalmazás célját
- átlátja a fő funkciókat
- képes önállóan használni a rendszert

Ez a megközelítés különösen fontos olyan rendszerek esetében, ahol a felhasználók informatikai ismeretei eltérőek lehetnek.

10.2 Az alkalmazás elérése

Az alkalmazás webalapú, vagyis **telepítést nem igényel**, kizárólag internetkapcsolatra és egy modern webböngészőre van szükség.
Ajánlott böngészők:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Az alkalmazás indításához a felhasználónak meg kell nyitnia a megadott webcímet, amely után automatikusan a főoldal jelenik meg.

10.3 A főoldal bemutatása

A főoldal az alkalmazás központi felülete.
Itt találhatóak meg azok az elemek, amelyek a rendszer funkcióihoz vezetik el a felhasználót.
A főoldalon:

- áttekinthető menürendszer található
- gyors elérés biztosított az adatkezelési funkciókhoz
- a felület letisztult és könnyen értelmezhető

A főoldal célja, hogy a felhasználó azonnal átlássa az alkalmazás használatának lehetőségeit.

10.4 Navigáció és menürendszer

Az alkalmazás navigációja egyszerű és logikus felépítésű.
A menüpontok elnevezései egyértelműek, így a felhasználó könnyen kiválaszthatja a kívánt műveletet.
A menürendszer előnyei:

- gyors navigáció
- egyértelmű funkcióelérés
- minimális tanulási idő

Ennek köszönhetően a rendszer használata nem igényel külön betanítást.

10.5 Adatok megtekintése

Az adatok megtekintése egy listázó felületen történik, ahol a rendszer a rögzített adatokat táblázatos formában jeleníti meg.
A listázó oldal jellemzői:

- áttekinthető adatszerkezet
- rendezett megjelenés
- műveleti lehetőségek minden rekordhoz

A felhasználó ezen a felületen könnyen áttekintheti a rendszerben szereplő adatokat.

10.6 Új adat rögzítése

Új adat rögzítése egy webes űrlap segítségével történik.
Az űrlapon található mezők az adatbázis szerkezetéhez igazodnak.
Az adatfelvitel lépései:

1. a megfelelő menüpont kiválasztása
2. az űrlap mezőinek kitöltése
3. az adatok elküldése

Sikeres adatfelvitel esetén a rendszer visszajelzést ad a művelet eredményéről.

10.7 Adatok módosítása

A rendszer lehetőséget biztosít a korábban rögzített adatok módosítására.
A szerkesztés egy előre kitöltött űrlapon történik, amely megkönnyíti a módosítás folyamatát.
A módosítás előnyei:

- gyors adatfrissítés
- hibák javítása
- naprakész adatkezelés

A felhasználó a módosítás után azonnal láthatja a frissített adatokat.

10.8 Adatok törlése

Az alkalmazás lehetőséget biztosít adatok törlésére is.
A törlés előtt a rendszer figyelmeztetést jelenít meg, amely megerősítést kér a felhasználótól.
Ez a megoldás:

- megakadályozza a véletlen törlést
- növeli az adatbiztonságot
- felhasználóbarát működést biztosít

A törlés végrehajtása után az adat véglegesen eltávolításra kerül a rendszerből.

10.9 Hibakezelés és visszajelzések

A rendszer minden fontos művelet esetén visszajelzést ad a felhasználónak.
Ez vonatkozik mind a sikeres, mind a sikertelen műveletekre.
A felhasználó számára megjelenhet:

- hibaüzenet hibás adatbevitel esetén

- megerősítés sikeres művelet után

Ez a működés segíti a felhasználót a rendszer helyes használatában.

10.10 A felhasználói dokumentáció összegzése

A felhasználói dokumentáció bemutatja az alkalmazás teljes használati folyamatát a rendszer elérésétől az adatok kezeléséig.
A leírás célja, hogy a felhasználó önállóan, segítség nélkül is képes legyen használni az alkalmazást.
A dokumentáció megfelel a szoftverfejlesztő–tesztelő képzés elvárásainak, és jól kiegészíti az alkalmazás technikai leírását.

11. Hibakezelés és validáció

11.1 A hibakezelés szerepe az alkalmazásban

A hibakezelés és validáció egy webalkalmazás egyik legfontosabb minőségi eleme. Feladata, hogy megakadályozza a hibás adatok feldolgozását, valamint megfelelő visszajelzést biztosítson a felhasználó számára.
A hibakezelés célja:

- az adatbázis védelme a hibás vagy hiányos adatoktól
- a felhasználó tájékoztatása a hibás műveletekről
- a rendszer stabil működésének fenntartása
- a felhasználói élmény javítása

A projekt során külön figyelmet kapott az, hogy a rendszer ne csak helyesen működjön, hanem **hibás használat esetén is kiszámítható és érthető legyen**.

11.2 Validáció fogalma és jelentősége

A validáció azon folyamatokat jelenti, amelyek ellenőrzik, hogy a felhasználó által megadott adatok megfelelnek-e a rendszer elvárásainak.
Validáció nélkül a rendszer sérülékeny lenne, és az adatok minősége hosszú távon romlana.
A validáció jelentősége:

- hibás adatbevitel megelőzése
- adatintegritás fenntartása
- backend terhelés csökkentése
- tesztelhetőség biztosítása

A validáció megvalósítása során a felhasználó minden esetben visszajelzést kap a hibák típusáról.

11.3 Kliensoldali validáció

A kliensoldali validáció a felhasználó böngészőjében történik, még mielőtt az adatok elküldésre kerülnének a szerver felé.
Ez gyors visszajelzést biztosít, és javítja a felhasználói élményt.
A kliensoldali validáció előnyei:

- azonnali visszajelzés
- kevesebb felesleges szerverkérés
- gyorsabb adatbevitel

A kliensoldali validáció azonban **nem helyettesíti** a szerveroldali validációt, kizárólag kiegészíti azt.

11.4 Szerveroldali validáció

A szerveroldali validáció az alkalmazás backend részén történik, és minden adatfeldolgozás előtt lefut.
Ez biztosítja, hogy a rendszer akkor is védett maradjon, ha a kliensoldali ellenőrzés megkerülésre kerül.
A szerveroldali validáció feladata:

- az adatbázis védelme
- üzleti logika érvényesítése
- biztonság növelése

A projekt során a szerveroldali validáció garantálja, hogy csak megfelelő adatok kerüljenek mentésre.

11.5 Validációs szabályok alkalmazása

Az alkalmazás különböző validációs szabályokat alkalmaz az adatok jellegének megfelelően.

Ezek a szabályok meghatározzák, hogy mely mezők kötelezőek, milyen hosszúságú adat engedélyezett, illetve milyen adattípus fogadható el.
Tipikus validációs szempontok:

- kötelező mezők ellenőrzése
- szöveghossz korlátozása
- számértékek ellenőrzése
- üres mezők kizárása

A szabályok következetes alkalmazása biztosítja az egységes adatkezelést.

11.6 Hibák megjelenítése a felhasználó számára

A hibák megjelenítése során kulcsfontosságú, hogy a felhasználó számára **érthető és egyértelmű üzenetek** jelenjenek meg.
A rendszer kerüli a technikai jellegű hibaüzeneteket, helyettük felhasználóbarát visszajelzéseket biztosít.
A visszajelzések célja:

- a hiba okának közlése
- a javítás módjának segítése
- a frusztráció csökkentése

Ez hozzájárul ahhoz, hogy a felhasználó könnyebben és magabiztosabban használja az alkalmazást.

11.7 Hibakezelés CRUD műveletek során

A hibakezelés minden adatkezelési művelet során fontos szerepet tölt be.
A rendszer külön kezeli a létrehozás, módosítás és törlés során felmerülő hibákat.
Példák:

- hiányos adatbevitel esetén figyelmeztetés jelenik meg
- nem létező rekord módosítása nem engedélyezett
- törlés előtt megerősítés szükséges

Ez a megközelítés csökkenti az adatvesztés kockázatát.

11.8 Hibakezelés és tesztelhetőség kapcsolata

A jól kialakított hibakezelés megkönnyíti a tesztelési folyamatot is.
A tesztelés során egyértelműen ellenőrizhető, hogy a rendszer hogyan reagál különböző hibás bemenetekre.
A tesztelés során vizsgált szempontok:

- hibaüzenetek megjelenése
- hibás adatok elutasítása
- rendszer stabilitása

Ez a fejezet közvetlenül kapcsolódik a tesztelési dokumentációhoz.

11.9 Biztonsági szempontok a hibakezelésben

A hibakezelés során a rendszer nem ad ki érzékeny technikai információkat.
Ez csökkenti a potenciális biztonsági kockázatokat.
Biztonsági megoldások:

- általános hibaüzenetek használata
- részletes technikai információk rejtve tartása
- naplózás csak fejlesztői szinten

Ez a megoldás megfelel a biztonságos szoftverfejlesztés alapelveinek.

11.10 Összegzés

A hibakezelés és validáció megvalósítása során a rendszer stabil, biztonságos és felhasználóbarát működést biztosít.

A kialakított megoldások hozzájárulnak az alkalmazás megbízhatóságához és tesztelhetőségéhez.
Ez a megközelítés megfelel a szoftverfejlesztő–tesztelő képzés elvárásainak, és jól illeszkedik a projekt teljes felépítéséhez.

12. TESZTELÉSI DOKUMENTÁCIÓ

(tesztelő képzésnél ez kiemelten fontos)

12.1 Tesztelési stratégia

A tesztelés manuálisan történt, a funkcionális követelmények alapján.

Tesztelt funkciók:

- adatfelvitel
- adat módosítás
- adat törlés
- hibás adatbevitel kezelése
- validációs üzenetek megjelenése

12.2 Tesztesetek (példa)

Teszteset 1

- Bemenet: üres mezők
- Elvárt eredmény: hibaüzenet
- Eredmény: megfelelően működik

Teszteset 2

- Bemenet: helyes adatok
- Elvárt eredmény: adat mentése
- Eredmény: sikeres

13. Verziókezelés és munkafolyamat

13.1 A verziókezelés szerepe a szoftverfejlesztésben

A verziókezelés a modern szoftverfejlesztés egyik alapvető eleme, amely lehetővé teszi a fejlesztési folyamat nyomon követését, a változások visszakereshetőségét, valamint az együttműködést több fejlesztő között is.
A verziókezelés fő céljai:

- a forráskód változásainak dokumentálása
- a hibás módosítások visszavonhatósága
- a fejlesztési folyamat átláthatósága
- a tesztelés támogatása

A projekt során alkalmazott verziókezelés biztosította, hogy a fejlesztés strukturált, ellenőrizhető és visszakövethető módon történjen.

13.2 Alkalmazott verziókezelő rendszer

A projekt során a **Git** verziókezelő rendszer került alkalmazásra, amely napjaink egyik legelterjedtebb és legmegbízhatóbb megoldása.
A Git előnyei:

- elosztott verziókezelés
- gyors működés
- rugalmas munkafolyamat-kezelés
- széles körű ipari elfogadottság

A Git lehetővé tette, hogy a fejlesztés minden egyes lépése külön rögzítésre kerüljön, így a projekt fejlődése pontosan nyomon követhető.

13.3 GitHub mint verziókezelési platform

A projekt forráskódja a **GitHub** platformon került tárolásra.
A GitHub nem csupán kódtárolásra szolgál, hanem hatékony eszközöket biztosít a fejlesztési folyamat dokumentálására is.
A GitHub használatának előnyei:

- távoli repository biztosítása
- projekt nyilvános vagy privát tárolása
- commit előzmények áttekintése
- fejlesztési tevékenység bemutatása

Vizsgaprojekt esetén a GitHub különösen előnyös, mivel hitelesen igazolja a fejlesztés önálló jellegét és folyamatát.

13.4 Commit-alapú munkavégzés

A fejlesztés során a módosítások **commitokra bontva** kerültek rögzítésre.
Minden commit egy jól meghatározott funkcionális változást képvisel.
A commitok jellemzői:

- világos, egyértelmű leírás
- egy funkció vagy javítás egy commitban
- fokozatos haladás dokumentálása

Ez a megközelítés segítette a hibák gyors beazonosítását, valamint megkönnyítette a visszatérést egy korábbi stabil állapothoz.

13.5 Fejlesztési munkafolyamat

A projekt fejlesztése **lépésről lépésre**, logikusan felépített munkafolyamat szerint történt.
A fejlesztési ciklus az alábbi fő szakaszokra bontható:

1. követelmények áttekintése

2. adatbázis-tervezés
3. backend funkciók megvalósítása
4. frontend felület kialakítása
5. hibajavítás és finomhangolás
6. tesztelés
7. dokumentáció elkészítése

Ez a struktúra biztosította, hogy a fejlesztés folyamatosan kontroll alatt maradjon.

13.6 Verziókezelés és tesztelés kapcsolata

A verziókezelés szorosan kapcsolódik a tesztelési folyamathoz.
A commitok segítségével pontosan nyomon követhető, hogy egy adott funkció mikor és milyen formában került bevezetésre.
A tesztelés során:

- új funkciók után tesztelés történt
- hibák javítása külön commitban jelent meg
- a működő állapotok megőrizhetők voltak

Ez jelentősen hozzájárult a rendszer stabilitásához.

13.7 Hibajavítás és változáskezelés

A fejlesztés során felmerülő hibák javítása strukturált módon történt.
A hibajavítások külön commitokban kerültek rögzítésre, így azok egyértelműen elkülönültek az új funkcióktól.
Ez a gyakorlat:

- javítja az átláthatóságot
- megkönnyíti a későbbi hibakeresést
- támogatja a tesztelési folyamatot

A változások kezelése így jól dokumentált és ellenőrizhető maradt.

13.8 Verziókezelés szerepe a vizsgaprojektben

A vizsgaremek esetében a verziókezelés nem csupán technikai, hanem szakmai értéket is képvisel.
A GitHub repository bemutatja:

- a fejlesztési folyamat lépéseit
- az egyes funkciók megvalósítását
- a hallgató önálló munkáját

Ez a dokumentált fejlesztés erősíti a projekt hitelességét és szakmai színvonalát.

13.9 Összegzés

A verziókezelés és az átgondolt munkafolyamat jelentősen hozzájárult a projekt sikeréhez.
A Git és a GitHub használata biztosította a fejlesztés átláthatóságát, visszakövethetőségét és tesztelhetőségét.
Az alkalmazott megoldások megfelelnek a szoftverfejlesztő–tesztelő képzés szakmai elvárásainak, és jól tükrözik a modern fejlesztési gyakorlatokat.

14. Biztonsági megoldások

14.1 A biztonság szerepe a webalkalmazásban

A webalkalmazások biztonsága kiemelten fontos szempont, mivel az alkalmazás felhasználói adatokat kezel és adatbázissal kommunikál.
A megfelelő biztonsági megoldások hiánya adatvesztéshez, jogosulatlan hozzáféréshez vagy a rendszer működésének veszélyeztetéséhez vezethet.
A projekt során a biztonság nem utólagos szempontként jelent meg, hanem már a tervezési fázistól kezdve beépült a rendszer működésébe.
Az alkalmazás célja nemcsak a helyes működés, hanem a **biztonságos működés** biztosítása is volt.

14.2 Felhasználói bemenetek biztonságos kezelése

A leggyakoribb biztonsági problémák forrása a felhasználó által megadott adatok nem megfelelő kezelése.
Ennek elkerülése érdekében az alkalmazás minden adatbevitel esetén ellenőrzéseket végez.
A bemenetek biztonságos kezelése során:

- csak elvárt típusú adatok kerülhetnek feldolgozásra
- a hibás vagy hiányos adatok elutasításra kerülnek
- a rendszer nem engedélyezi a veszélyes adatbeviteleket

Ez a megközelítés csökkenti a rosszindulatú beavatkozások lehetőségét és növeli a rendszer stabilitását.

14.3 SQL Injection elleni védelem

Az SQL Injection az egyik legismertebb webes támadási forma, amely során a támadó az adatbázis-lekérdezések manipulálására törekszik.
A projekt során ez a kockázat hatékony védelemmel lett kezelve.
A Laravel keretrendszer és az **Eloquent ORM** használata alapértelmezetten védi az alkalmazást az SQL Injection támadásokkal szemben.
Az adatbázisműveletek objektumorientált módon történnek, így a felhasználói adatok nem közvetlenül kerülnek be az SQL utasításokba.
Ez jelentősen növeli az adatbázis biztonságát.

14.4 CSRF (Cross-Site Request Forgery) elleni védelem

A CSRF támadás során egy külső oldal próbál jogosulatlan műveletet végrehajtani egy hitelesített felhasználó nevében.
A Laravel beépített CSRF védelmet biztosít minden űrlap esetében.
A CSRF védelem működése:

- minden űrlap egyedi tokent kap
- a token ellenőrzésre kerül a kérés feldolgozása előtt
- érvénytelen token esetén a kérés elutasításra kerül

Ez biztosítja, hogy csak az alkalmazás által létrehozott, érvényes kérések kerüljenek feldolgozásra.

14.5 Hibakezelés biztonsági szempontból

A hibakezelés nemcsak funkcionalitási, hanem biztonsági kérdés is.
A rendszer kialakítása során fontos szempont volt, hogy a hibaüzenetek ne tartalmazzanak érzékeny technikai információkat.
Biztonsági megoldások a hibakezelés során:

- általános hibaüzenetek megjelenítése
- részletes technikai hibák elrejtése a felhasználó elől
- fejlesztői információk naplózása külön szinten

Ez megakadályozza, hogy egy támadó információkat gyűjtsön a rendszer belső működéséről.

14.6 Jogosultságok és műveletek védelme

Az alkalmazás adatkezelési műveletei ellenőrzött módon hajthatók végre.
A rendszer biztosítja, hogy a műveletek csak a megfelelő feltételek mellett történjenek meg.
Biztonsági célok:

- jogosulatlan műveletek megakadályozása
- véletlen adatvesztés elkerülése
- adatkezelési folyamatok kontrollálása

Ez a struktúra hozzájárul az alkalmazás megbízhatóságához és átláthatóságához.

14.7 Adatok integritásának biztosítása

Az adatbázis-szintű és alkalmazás-szintű ellenőrzések együttesen biztosítják az adatok integritását.
A rendszer nem engedi meg a követelményeknek nem megfelelő adatok mentését.
Az adatbiztonságot erősíti:

- szigorú validáció
- strukturált adatbázis-felépítés
- ellenőrzött CRUD műveletek

Ez csökkenti az adatsérülések kockázatát és növeli a rendszer megbízhatóságát.

14.8 Biztonság és tesztelés kapcsolata

A biztonsági megoldások működését a tesztelési folyamat során is ellenőriztük.
A tesztelés során vizsgálatra került, hogy a rendszer megfelelően reagál-e hibás vagy nem megengedett műveletekre.
Tesztelt biztonsági szempontok:

- hibás adatbevitel elutasítása
- jogosulatlan műveletek kezelése
- stabil működés rendellenes használat esetén

A teszteredmények igazolták, hogy az alkalmazás biztonsági megoldásai megfelelően működnek.

14.9 A biztonsági megoldások szerepe a vizsgaprojektben

A biztonság megfelelő kezelése fontos része egy szoftverfejlesztő–tesztelő vizsgaprojektnek.
A projekt során alkalmazott megoldások bizonyítják, hogy a fejlesztés során nemcsak a funkcionalitás, hanem a minőség és megbízhatóság is fontos szempont volt.
Ez a fejezet jól bemutatja a hallgató biztonságtudatos szemléletét.

14.10 Összegzés

A biztonsági megoldások kialakítása során a rendszer stabil, megbízható és védett működést biztosít.
Az alkalmazott fejlesztési elvek és technológiák megfelelnek a modern webalkalmazásokkal szemben támasztott elvárásoknak.
A megvalósított biztonsági megoldások illeszkednek a szoftverfejlesztő–tesztelő képzés szakmai követelményeihez, és hozzájárulnak a projekt sikeréhez.

15. Telepítési útmutató

15.1 A telepítési útmutató célja

A telepítési útmutató célja, hogy részletes, lépésről lépésre követhető leírást adjon az alkalmazás futtatásához szükséges környezet beállításáról.
Ez a fejezet biztosítja, hogy az alkalmazás egy másik számítógépen is könnyedén üzembe helyezhető legyen.
A dokumentáció célközönsége:

- oktatók és vizsgabizottság tagjai
- fejlesztők
- rendszert üzemeltető személyek

Az útmutató bemutatja a szükséges szoftvereket, azok beállítását, valamint az alkalmazás indításának módját.

15.2 Rendszerkövetelmények

Az alkalmazás futtatásához az alábbi alapvető hardver- és szoftverfeltételek szükségesek:
Hardver követelmények:

- asztali számítógép vagy laptop
- legalább 4 GB RAM
- stabil internetkapcsolat

Szoftver követelmények:

- Windows vagy Linux operációs rendszer
- modern webböngésző (Chrome, Firefox, Edge)

Ezek a követelmények biztosítják az alkalmazás zavartalan működését fejlesztési környezetben.

15.3 Fejlesztői környezet előkészítése

A projekt helyi futtatásához egy **lokális fejlesztői környezet** szükséges.
Erre a célra a **XAMPP** környezet került alkalmazásra, amely egy csomagban tartalmazza a szükséges szerveroldali komponenseket.
A XAMPP tartalmazza:

- Apache webszervert
- PHP értelmezőt
- MySQL adatbáziskezelőt

A XAMPP telepítése után az Apache és MySQL szolgáltatásokat el kell indítani a vezérlőpulton.

15.4 A projekt forráskódjának beszerzése

Az alkalmazás forráskódja verziókezelő rendszerben, GitHubon található meg.
A projekt letöltése történhet:

- repository klónozásával
- vagy tömörített archív fájl letöltésével

A letöltött projektet a webszerver dokumentum gyökerében kell elhelyezni, hogy az Apache szerver el tudja érni azt.

15.5 Fájlok és mappák elhelyezése

A Laravel projekt mappaszerkezete előre definiált.
Fontos, hogy a projekt valamennyi fájlja sértetlen állapotban kerüljön a telepítési könyvtárba.
A struktúra megfelelő elhelyezése biztosítja:

- a routing helyes működését
- a nézetek betöltését
- az adatbáziskapcsolat működését

Hibás mappastruktúra esetén az alkalmazás nem indul el megfelelően.

15.6 Függőségek telepítése
A Laravel projekt működéséhez külső csomagok szükségesek, amelyeket a **Composer** kezel.
A projekt főkönyvtárában a függőségek telepíthetők a megfelelő parancs kiadásával.
A függőségek telepítése után az alkalmazás minden szükséges könyvtárral rendelkezik a futáshoz.
Ez a lépés kulcsfontosságú a rendszer megfelelő működése szempontjából.

15.7 Környezeti beállítások
A Laravel alkalmazás konfigurációja egy környezeti beállításokat tartalmazó fájlon keresztül történik.
Ebben a fájlban kerülnek megadásra az adatbázis-kapcsolathoz szükséges adatok.
A helyes beállítások nélkül:

- az adatbázis nem érhető el
- az alkalmazás hibát jelezhet
- az adatok mentése sikertelen lehet

A konfiguráció ellenőrzése ezért a telepítés egyik legfontosabb lépése.

15.8 Adatbázis létrehozása és beállítása
A MySQL adatbázis létrehozása a helyi adatbáziskezelő eszköz segítségével történik.
A létrehozott adatbázishoz a megfelelő felhasználói jogosultságokat biztosítani kell.
Az adatbázis szerkezete Laravel migrációk segítségével hozható létre, amely garantálja az egységes struktúrát és a verziókezelhetőséget.

15.9 Az alkalmazás indítása
A telepítés befejezése után az alkalmazás elindítható a böngésző segítségével.
A helyes URL megadását követően az alkalmazás főoldala megjelenik.
Sikeres indítás esetén:

- a főoldal betöltődik
- az adatlista elérhető
- az űrlapok működnek

Ez jelzi, hogy a telepítés sikeresen lezajlott.

15.10 Gyakori problémák és megoldások
A telepítés során előfordulhatnak kisebb problémák, amelyek általában konfigurációs hibákra vezethetők vissza.
Gyakori hibák:

- adatbázis-kapcsolati hiba
- hiányzó függőségek
- jogosultsági problémák

Ezek a problémák a beállítások ellenőrzésével és a telepítési lépések ismételt áttekintésével általában gyorsan megoldhatók.

15.11 Összegzés
A telepítési útmutató részletesen bemutatja az alkalmazás helyi környezetben történő futtatásának teljes folyamatát.
A leírt lépések követésével az alkalmazás biztonságosan és megbízható módon üzembe helyezhető.
Ez a fejezet hozzájárul ahhoz, hogy a vizsgaprojekt teljes értékű, dokumentált és önálló munkaként legyen értékelhető.

16. Továbbfejlesztési lehetőségek

16.1 A továbbfejlesztés jelentősége

A szoftverfejlesztés ritkán zárul le egyetlen verzió elkészítésével.
A legtöbb alkalmazás folyamatosan fejlődik, új funkciókkal bővül, illetve a felhasználói igényekhez igazodik.
Ezért fontos, hogy már a projekt elkészítése során felmerüljenek a továbbfejlesztési lehetőségek.
Jelen vizsgaremek esetében az alkalmazás stabil alapot biztosít, amelyre további funkciók és bővítések építhetők.
A következő alfejezetek bemutatják azokat a fejlesztési irányokat, amelyekkel a rendszer továbbfejleszthető lenne.

16.2 Felhasználókezelés és jogosultsági rendszer

Az egyik legkézenfekvőbb továbbfejlesztési lehetőség a felhasználókezelés bevezetése.
Ez lehetővé tenné, hogy a rendszer több felhasználót különböztessen meg, valamint eltérő jogosultságokat biztosítson számukra.
Lehetséges funkciók:

- regisztráció és bejelentkezés
- felhasználói szerepkörök (pl. admin, felhasználó)
- jogosultság alapú műveletvégzés

Ezzel a fejlesztéssel az alkalmazás biztonságosabbá és skálázhatóbbá válna.

16.3 Keresési és szűrési funkciók

A jelenlegi adatkezelő rendszer továbbfejleszthető keresési és szűrési lehetőségekkel.
Ez különösen nagyobb adatmennyiség esetén növeli a használhatóságot.
Példák:

- szöveges keresés adatok között
- szűrés különböző mezők szerint
- rendezés növekvő vagy csökkenő sorrendben

Ezek az eszközök jelentősen javítanák a felhasználói élményt.

16.4 Automatikus tesztelés bevezetése

A projekt során manuális tesztelés történt, azonban a rendszer továbbfejleszthető automatikus teszteléssel is.
Ez különösen fontos lehet egy nagyobb, komplexebb rendszer esetén.
Lehetséges teszttípusok:

- egységtesztek
- integrációs tesztek
- funkcionális tesztek

Az automatikus tesztelés csökkenti az emberi hibák számát, és gyorsabb hibafelismerést tesz lehetővé.

16.5 Adatvizualizáció és statisztikák

A rendszer továbbfejleszthető különböző statisztikai és adatvizualizációs funkciókkal.
Ez lehetővé tenné az adatok grafikus megjelenítését.
Példák:

- diagramok
- kimutatások
- összesített statisztikák

Az ilyen jellegű fejlesztések informatívabbá tennék az alkalmazást, különösen üzleti környezetben.

16.6 Reszponzív felület továbbfejlesztése

Bár az alkalmazás már jelenleg is reszponzív, további finomhangolás történhetne mobil- és tablet-nézetek esetében.
Ez növelné a felhasználhatóságot különböző eszközökön.
Lehetséges fejlesztések:

- mobilbarát menürendszer
- érintőképernyő-optimalizált gombok
- sebességoptimalizálás kisebb eszközökön

16.7 REST API kialakítása

Egy további fejlesztési irány lehet egy REST API kifejlesztése, amely lehetővé tenné az alkalmazás adatainak külső rendszerekkel való összekapcsolását.
Az API előnyei:

- más alkalmazásokkal való integráció
- mobilalkalmazás fejlesztésének lehetősége
- skálázható rendszer kialakítása

Ez a fejlesztés már ipari környezetben is jól hasznosítható lenne.

16.8 Naplózás és monitorozás

A rendszer továbbfejleszthető naplózási és monitorozási funkciókkal.
Ez segítené a hibák gyorsabb felismerését és az alkalmazás állapotának nyomon követését.
Lehetséges megoldások:

- műveletek naplózása
- hibák rögzítése
- statisztikák a használatról

Ez a fejlesztés különösen fontos hosszú távú üzemeltetés esetén.

16.9 A továbbfejlesztések szerepe a vizsgaprojektben

A továbbfejlesztési lehetőségek bemutatása bizonyítja, hogy a hallgató átfogóan gondolkodik a szoftverfejlesztésről.
Nemcsak egy működő alkalmazás elkészítése volt a cél, hanem egy olyan rendszer létrehozása, amely a jövőben tovább bővíthető.
Ez a szemlélet megfelel a szoftverfejlesztő–tesztelő képzés szakmai elvárásainak.

16.10 Összegzés

A bemutatott továbbfejlesztési lehetőségek jól mutatják, hogy az alkalmazás stabil alapokra épül és számos irányban bővíthető.
A javasolt fejlesztések mind hozzájárulnának a rendszer funkcionalitásának, biztonságának és használhatóságának növeléséhez.
Ez a fejezet lezárja a projekt jövőbe mutató szemléletét, és megfelelően egészíti ki a vizsgaremek dokumentációját.

17. Összegzés

17.1 A projekt céljának teljesülése

A vizsgaremek elkészítésének célja egy olyan webalapú alkalmazás megvalósítása volt, amely megfelel az Oktáv Szoftverfejlesztő–tesztelő képzés szakmai és formai követelményeinek.
A projekt során egy Laravel alapú adatkezelő rendszer készült el, amely stabil, átlátható és jól dokumentált megoldást nyújt.
A kitűzött célok teljesültek, mivel az alkalmazás:

- MVC architektúrát alkalmaz
- adatbázissal kommunikál
- CRUD műveleteket valósít meg
- megfelelő hibakezeléssel és validációval rendelkezik
- tesztelhető és továbbfejleszthető

Ezáltal a projekt alkalmas vizsgaremekként történő benyújtásra.

17.2 Az alkalmazás szakmai értéke

A projekt elkészítése során számos, a szoftverfejlesztésben alapvetőnek számító szakmai terület került érintésre.
A fejlesztés nem kizárólag a működő alkalmazás létrehozására fókuszált, hanem a mögöttes elvek, folyamatok és dokumentáció bemutatására is.
A vizsgaremek szakmai értékét növeli:

- a modern Laravel keretrendszer használata
- strukturált adatbázis-tervezés
- frontend és backend elkülönítése
- verziókezelés alkalmazása
- biztonsági megoldások beépítése

Ezek a megoldások jól tükrözik a hallgató felkészültségét a szoftverfejlesztő–tesztelő szerepkörre.

17.3 Tanulási tapasztalatok

A projekt elkészítése során a hallgató számos új ismeretet és gyakorlati tapasztalatot szerzett.
A fejlesztési folyamat lehetőséget biztosított a tanult technológiák mélyebb megértésére és gyakorlati alkalmazására.
A legfontosabb tanulási tapasztalatok:

- komplex rendszer lépésről lépésre történő megvalósítása
- hibák felismerése és javítása
- dokumentáció fontosságának megtapasztalása
- rendszeres tesztelés jelentősége
- verziókezelés szerepe a fejlesztésben

Ezek a tapasztalatok hozzájárulnak a szakmai fejlődéshez és a későbbi munkaerőpiaci elhelyezkedéshez.

17.4 Tesztelési és minőségbiztosítási szemlélet

A vizsgaremek elkészítése során kiemelt szerepet kapott a tesztelés és a minőségbiztosítás.
A manuális tesztelés lehetővé tette a funkciók részletes ellenőrzését és a hibás bemenetek megfelelő kezelésének vizsgálatát.
A tesztelési szemlélet eredménye:

- stabil működés
- kiszámítható rendszerreakciók
- megbízható adatkezelés

Ez a megközelítés megfelel a szoftverfejlesztő–tesztelő képzés elvárásainak.

17.5 A dokumentáció szerepe

A projekt dokumentációja részletesen bemutatja az alkalmazás felépítését, működését és használatát.
A dokumentáció célja nemcsak a vizsgára való felkészítés, hanem egy olyan átfogó leírás létrehozása, amely később is hasznos lehet.
A dokumentáció tartalmazza:

- technikai leírásokat
- felhasználói útmutatót
- tesztelési dokumentációt
- telepítési útmutatót
- továbbfejlesztési lehetőségeket

Ez a komplex dokumentáció hozzájárul a projekt professzionális megítéléséhez.

17.6 Összhang az Oktáv képzés követelményeivel

A vizsgaremek minden szempontból illeszkedik az Oktáv Szoftverfejlesztő–tesztelő képzés követelményrendszeréhez.
A projekt bemutatja a tanult technológiák alkalmazását, a fejlesztési folyamatot és a tesztelési szemléletet.
A vizsgaremek:

- megfelel a szakmai elvárásoknak
- dokumentált és áttekinthető
- önálló munkát tükröz
- gyakorlati tudást mutat be

Ezáltal alkalmas a képzés sikeres lezárására.

17.7 Záró gondolatok

Összességében elmondható, hogy a projekt sikeresen teljesíti a vizsgaremekkel szemben támasztott elvárásokat.
Az elkészült alkalmazás stabil alapot biztosít a további fejlesztésekhez, valamint jól bemutatja a hallgató szakmai felkészültségét.
A vizsgaremek elkészítése hozzájárult a szoftverfejlesztő–tesztelő kompetenciák fejlődéséhez, és megfelelő zárása az Oktáv képzés során szerzett ismereteknek.

18. FORRÁSJEGYZÉK

- Laravel hivatalos dokumentáció – https://laravel.com/docs
- PHP dokumentáció – https://www.php.net/docs.php
- Bootstrap – https://getbootstrap.com
- MySQL – https://dev.mysql.com/doc/